

Contents

Enterprise AI for Information Systems Faculty	2
Slide companion — expanded notes	2
Part 1 — The frame	2
The through-line	2
Generative AI is another layer — not a teardown	2
The 7-layer enterprise AI stack (the map)	3
Part 2 — Layer 1: Experience	3
Where users meet the AI	3
Part 3 — Layer 2: Orchestration	4
What turns a model into a system that acts	4
The agent loop	4
MCP — connecting the model to tools and data	4
A2A — agents that delegate to each other	5
Keeping multi-step actions atomic (two-phase commit)	5
Agentic patterns	5
Putting it together: a modern agent architecture	5
Part 4 — Layer 3: Model	6
Foundation model anatomy	6
The adaptation ladder	6
Part 5 — Layer 4: Retrieval & context	6
Grounding is more than a vector database	6
Prompt → retrieval → tool-use (the grounding lab)	7
Part 6 — Layer 5: Enterprise systems	7
What the AI reads — and acts on	7
Part 7 — Layer 6: Data foundation	7
The ground truth everything stands on	7
Part 8 — Layer 7: Governance & observability (cross-cutting)	8
How agents fail	8
Build → attack → govern	8
Governing agents = IS governance, specialized	9
The Agent Risk & Governance Canvas	9
Part 9 — Closing cases	9
Case 1 — Customer refund assistant	9
Case 2 — HR policy assistant	10
Case 3 — Monthly variance-report agent	10
Part 10 — Live demos (run them yourself)	10
Appendix — How this maps to the two MS programs	11

Enterprise AI for Information Systems Faculty

Slide companion — expanded notes

Prepared by: Tim Smith · Companion to the interactive deck *Enterprise AI for IS Faculty*

This document expands on every slide in the deck. It is meant to be read alongside the slides — as a refresher before you present, a leave-behind for attendees, or raw material when you translate a topic into your own course. The organizing idea is a single **7-layer enterprise AI stack**; each section below maps to a slide and deepens it, with teaching notes and ties to the two new MS programs where useful.

A note on scope: the goal is conceptual and organizational fluency, not machine-learning research depth. Everything here is pitched at a senior IS faculty member who needs to teach, advise, and lead AI-centered curriculum work credibly.

Part 1 — The frame

The through-line

Every layer in this deck answers one question: **how do AI systems move from impressive demos to reliable, secure, governable enterprise capabilities?** This is the right question for IS faculty because the journey from demo to production is exactly the territory the discipline already owns — requirements, data management, integration, security, governance, and change management.

A working demo proves that a capability is *possible*. Production proves it is *dependable, safe, and economical at scale*. The gap between the two is where most enterprise AI initiatives stall, and it is almost never a model-quality problem. It is a data, integration, evaluation, and governance problem — IS problems.

Teaching note. Open with this question and return to it at every layer. It reframes “AI anxiety” as familiar engineering and management work seen through a new lens.

Generative AI is another layer — not a teardown

Generative AI sits *on top of* the analytics and data stack faculty already teach; it does not replace it. Traditional machine learning (regression, trees, clustering, deep learning) still does most predictive work in enterprises. Generative AI adds a new layer that is good at language, synthesis, and orchestration.

Framing GenAI as an additional layer accomplishes two things. It pre-empts the “is everything we taught now obsolete?” reaction, and it lets us organize the entire subject as a **stack** — which is the backbone of this deck.

Teaching note. When a colleague worries their course is now irrelevant, locate their course on the stack (most live at the data, analytics, architecture, or governance layers) and show how it *grounds* the GenAI layer rather than competing with it.

The 7-layer enterprise AI stack (the map)

The stack is the single map used throughout the deck. Briefly, top to bottom:

1. **Experience** — where users meet the AI: chat, copilots, embedded assistants. Trust and verifiability must surface here.
2. **Orchestration** — what turns a model into a system that acts: agents, workflows, routing, state, approval gates, plus MCP and A2A. Most of the engineering and risk lives here.
3. **Model** — the reasoning engine: LLM/SLM choice, prompting, fine-tuning/LoRA, adapters.
4. **Retrieval & context** — grounding the model in your facts: hybrid + graph + SQL search, MCP, context engineering — much more than a vector database.
5. **Enterprise systems** — what the AI reads and acts on: ERP, CRM, business apps and APIs. Every tool is a capability and a risk.
6. **Data foundation** — the ground truth: warehouses, lakes, pipelines, and quality. A recurring root cause of AI project failure (RAND, 2024 — see References).
7. **Governance & observability** — security, RBAC, audit, evaluation, and compliance. This layer **cross-cuts** all the others; it is never bolted on at the end.

Teaching note. Each course in the two MS programs lands on one or two of these layers. Asking faculty to place their course on the stack is the fastest way to surface gaps and overlaps in the curriculum.

Part 2 — Layer 1: Experience

Where users meet the AI

The experience layer is the surface users actually touch, and its design choices determine whether an AI system is trusted or quietly abandoned. Three common shapes:

- **Standalone chat** — a direct assistant. Lowest friction to build, and the easiest to misuse when answers are not grounded.
- **Embedded copilot** — AI inside an existing application (CRM, IDE, BI tool). It meets users in context and inherits the host application’s data and permissions.
- **Background agent** — software that acts without a chat window; its “interface” is a notification or an approval request.

The design rule that matters most: **verifiability has to surface at this layer**. Citations, a visible “show your work” trail, and approval prompts for risky actions are experience-layer features even though they depend on every layer beneath them. An answer the user cannot check is a liability no matter how good the model is.

Teaching note. Have students critique a real AI feature purely on its experience layer: can the user tell where the answer came from, and can they intervene before something irreversible happens?

Part 3 — Layer 2: Orchestration

What turns a model into a system that acts

A model on its own produces text. Orchestration is the engineering that wires the model to tools, state, memory, and humans so it can accomplish work reliably. Be precise about three levels, because **risk scales with autonomy**:

- **Chatbot** — answers in one shot. No tools, no actions. Low autonomy, low risk.
- **AI-augmented workflow** — fixed steps with AI inside some of them. *You* hold the control flow. Medium autonomy.
- **Agent** — the system itself decides which tools to call and when, looping until the goal is met. High autonomy, high risk.

Only the third is genuinely “agentic.” Much of the market hype blurs these together; teaching students to name which one they are actually building is a high-leverage clarity move.

The agent loop

An agent runs a loop: **plan** → (**approval gate**) → **act** → **observe** → **repeat**, until the goal is met. Memory persists across the iterations so the agent can build on what it has already learned in the task.

- **Plan** — decide the next step toward the goal.
- **Approval gate** — for risky or irreversible actions, a human approves before the agent acts. This is optional but essential for high-stakes tools.
- **Act** — call a tool (query a database, send an email, run code).
- **Observe** — read the result and feed it back into the next plan.

The loop is simple; the discipline is in deciding which actions need a gate, how many iterations are allowed before the agent gives up, and what is recorded along the way.

Teaching note. Most “the agent went rogue” stories are really “the loop had no gate and no iteration cap.” Frame those controls as ordinary engineering, not exotic AI safety.

MCP — connecting the model to tools and data

The Model Context Protocol (MCP) is an open standard for connecting a model (the *client*, embedded in a host application) to a *server* you write, which exposes your tools and data. The model asks the server to do things; the server decides what is allowed and returns only results.

Two properties make this matter for enterprises:

- **Decoupling.** You can swap the underlying model without rewriting your tools, because both sides speak the same protocol. That avoids vendor lock-in.
- **A trust boundary.** Credentials and raw data stay on the server side. The model only ever sees what the server chooses to return — it never holds your database password or sees rows it is not permitted to see.

This is exactly what Varol’s hands-on session builds, so the deck deliberately frames MCP as the connective tissue rather than re-teaching the build.

A2A — agents that delegate to each other

As systems grow, a single agent is often replaced by a team: an **orchestrator** delegates sub-tasks to **specialist** agents (a research agent, a SQL/data agent, a report agent), each owning its own tools. Agent-to-agent (A2A) communication is how they discover each other, hand off work, and report back.

The new question this raises — and a genuinely good IS exam question — is **trust and accountability between autonomous agents**. If agent A acts on agent B's output and the result is wrong, who is accountable, and how is the chain auditable?

Keeping multi-step actions atomic (two-phase commit)

When an agent must do several consequential things together — for example, *reserve inventory and charge a card* — partial success is dangerous. You never want a charged card with no reserved stock. **Two-phase commit (2PC)** is the classic coordination protocol that prevents this:

1. **Phase 1 — Prepare.** The coordinator asks every participant whether it *can* commit. Each locks its resources and votes YES or NO.
2. **Decision.** If all vote YES, the coordinator decides COMMIT; if any votes NO (or times out), it decides ABORT.
3. **Phase 2 — Commit / Abort.** Every participant either finalizes or rolls back. Either way the system ends in a consistent state.

This is not new — it is core distributed-systems material — but it is suddenly relevant again because agents now trigger multi-system actions on their own. Orchestration is what keeps those actions atomic.

Teaching note. This is a direct bridge from existing database/distributed-systems courses into agentic AI. The hands-on stepper in the deck lets faculty inject a failure and watch the abort path.

Agentic patterns

Patterns outlive the frameworks that implement them, so teaching the pattern is the more durable curriculum move. Four worth knowing:

- **ReAct** — the agent interleaves reasoning and acting in a tight loop (think, act, observe, repeat).
- **Plan-and-execute** — a planner lays out the steps first, then they are executed in order.
- **Reflection / critique** — the agent drafts, critiques its own draft, and revises.
- **Multi-agent** — a lead agent coordinates several worker agents.

Whatever library is fashionable this year, it implements some combination of these. Teach the patterns; let the frameworks come and go.

Putting it together: a modern agent architecture

A realistic enterprise agent is a tool-using loop **plus** the pieces above: MCP for governed access to data and tools, an orchestrator with shared state (and 2PC for atomic actions), and optionally A2A peers. Each addition buys capability and adds a new risk surface:

- **MCP** buys swappable, governed access and a credential boundary; it adds tool/permission sprawl to manage.

- **Orchestration + state** buys atomic multi-step actions and central approval gates; it makes the coordinator a single chokepoint.
- **A2A** buys specialization and delegation; it adds inter-agent trust and cascading-failure risk.

The deck’s interactive lets you toggle these and watch the “what it buys / what it risks” lists change — a compact way to show that architecture decisions are always trade-offs.

Part 4 — Layer 3: Model

Foundation model anatomy

Two phases are worth separating because they explain different costs and behaviors.

Built once. A model is *pretrained* on web-scale text, then shaped by *instruction tuning* and *RLHF* (reinforcement learning from human feedback) into something that follows directions. The result is a foundation model with a fixed set of parameters and a finite **context window**.

Used every call. On each request, your prompt and any retrieved context are *tokenized*, run through the model, and a response is produced. The context window is the working-memory budget for that call — everything the model can “see” at once.

Keep this conceptual, not mathematical. Faculty need enough vocabulary to explain cost (tokens), latency, and the most common failure mode (asking the model to use information that never made it into the context window).

The adaptation ladder

There is a ladder of moves for specializing a general model to your work, and each rung adds capability **and** risk:

Prompt → **add examples** → **add retrieval** → **fine-tune** → **add tools / agency**.

The single most useful decision rule: **fine-tune for style, format, and domain behavior; use retrieval to give the model new facts**. Trying to teach a model new facts by fine-tuning is expensive, brittle, and goes stale — that is a retrieval problem (Layer 4), not a fine-tuning problem. Treating the options as a ladder, rather than as interchangeable, is sharper and prevents a lot of wasted effort.

Teaching note. A good assignment: give students a business need and have them justify the lowest rung that actually solves it. Most real needs are solved at “retrieval,” not “fine-tune.”

Part 5 — Layer 4: Retrieval & context

Grounding is more than a vector database

“RAG” is often taught as “embed documents, store vectors, search by similarity.” That is one method, not the whole picture. Modern retrieval **blends** several methods and then engineers the context the model actually receives:

- **Vector search** for semantic similarity.
- **Keyword / BM25** for exact terms, names, and codes that embeddings miss.
- **Knowledge graphs** for relationships and structured connections.
- **SQL / structured queries** for authoritative numbers from systems of record.
- **Live tools (via MCP)** for current, transactional data.

The results are merged, ranked, and assembled into the context the model sees — a discipline increasingly called **context engineering**. The headline for IS faculty: retrieval is fundamentally an *information-retrieval and data* problem, which is precisely why so much of it reuses existing data-management expertise.

Prompt → retrieval → tool-use (the grounding lab)

The deck’s interactive runs one question through three levels of structure and shows how the answer’s quality changes:

- **Prompt only** — fluent and plausible, but unverifiable. The model is guessing from training data with no link to your policy or this transaction.
- **+ Retrieval** — the answer is now grounded in your documents and quotable, though still blind to live data.
- **+ Tool use** — the answer is grounded, current, and traceable: a cited policy *plus* a live, logged tool call you can audit.

The three meters — grounding, verifiability, freshness — rise together as structure is added. The lesson is that reliability comes from architecture around the model, not from a better model alone.

Part 6 — Layer 5: Enterprise systems

What the AI reads — and acts on

The “tools” an agent can call are your business systems: ERP, CRM, internal APIs, file stores. This layer is where capability and risk become concrete, because every tool the agent can use is also part of its **blast radius**.

The crucial distinction is **read vs. write**. Reads are comparatively safe and cheap. Writes — creating, updating, deleting records in systems of record — are consequential and often irreversible. The design principle is **least privilege**: an agent’s tool allowlist *is* its blast radius, so scope it tightly, gate the writes, and log everything.

Teaching note. This connects directly to access control and enterprise architecture courses. “What is on this agent’s tool allowlist?” is the agentic version of “what can this role do?”

Part 7 — Layer 6: Data foundation

The ground truth everything stands on

Beneath everything is data: warehouses, lakes, pipelines, and the quality and permissions attached to them. The headline for this layer comes from a RAND study based on interviews with 65 data

scientists and engineers: **more than 80% of AI projects fail to reach production — about twice the failure rate of non-AI IT projects — with inadequate data a recurring root cause** (RAND, 2024; see References). The common data faults — bad chunking, stale documents, conflicting sources, missing metadata, permission leaks — are not *caused* by the retrieval pipeline; they are *exposed* by it. (RAND’s single most common cause is actually misunderstanding the project’s purpose, with data close behind — a useful caution against blaming the model.)

What “good” looks like here is unglamorous and familiar: data quality and lineage you can trust, freshness (re-indexing when sources change), permissions carried all the way through to retrieval, and pipelines that are observable rather than black boxes. This is the layer Don Berndt’s deep-dive expands, and it is where IS faculty’s existing data-management strength is most directly reusable.

Teaching note. When a GenAI project disappoints, debug *down* the stack. The problem is far more often at Layer 6 than at Layer 3.

Part 8 — Layer 7: Governance & observability (cross-cutting)

How agents fail

The hard part is production, not the demo, and the OWASP Top 10 for LLM Applications is a good anchor for the discussion of how systems break. The recurring failure modes:

- **Prompt injection** — untrusted content (a retrieved document, a user message) hijacks the instructions.
- **Excessive agency** — the agent has more capability than the task needs.
- **Tool misuse** — the right tool called in a wrong or dangerous way.
- **Cost blowout** — loops and retries nobody capped.
- **Accountability gaps** — no trace of *why* the system did what it did.
- **Data leakage** — sensitive data surfaced to the wrong party.

Build → attack → govern

The most effective way to teach this is hands-on: build a small agent, attack it, then apply controls. The deck’s lab pairs each attack with a control and shows whether it holds:

- **Prompt injection** is best stopped by **retrieval/input filtering** (quarantine untrusted content before the model sees it); other controls only reduce it.
- **Exfiltration** and **tool misuse** are best stopped by **RBAC / least-privilege tools** — if the agent has no permission to email externally or call an admin API, the action simply cannot happen.
- **Excessive agency** is best stopped by a **human approval gate** on irreversible actions, plus least privilege.
- **Audit logs and an evaluation harness** detect and flag, but rarely prevent — they are necessary, not sufficient.

The takeaway: **no single control is enough — defense in depth.**

Governing agents = IS governance, specialized

Agent governance is not a new discipline; it is an extension of governance IS faculty already teach. The mapping is almost one-to-one:

- **Tool permissioning** is the new **RBAC / least privilege**.
- **Action approval gates** are the new **change control**.
- **Reasoning and tool traces** are the new **audit logging**.
- **Non-deterministic testing** demands an **evaluation harness**, because deterministic test cases fail on probabilistic systems.

Anchor all of this to standards your courses already touch — **NIST AI RMF**, **ISO/IEC 42001**, **the EU AI Act** — mapped onto familiar frameworks like **COBIT** and **NIST CSF**. Observability is the enabler underneath: capture the full trace, evaluate continuously, audit on demand.

The Agent Risk & Governance Canvas

The canvas is a one-page tool for reasoning about a single agentic use case before it is built. Its nine cells: **use case & business value**, **stakeholders**, **data sources**, **model & adapter**, **tools & autonomous actions**, **human oversight points**, **security risks**, **compliance**, and **evaluation & trace observability**. Filling it in forces the hard questions early — especially “which actions are irreversible, and where is the human gate?” — and produces an artifact that feeds directly into the cases below and into students’ coursework.

Part 9 — Closing cases

Each case follows the same shape so the structure itself becomes a teaching tool: **Problem** → **Solution (across the stack)** → **Monitoring & maintenance** → **Governance**. The layer rail on each case slide shows exactly which of the seven layers the solution spans. The point is to show that a real system is never one layer — it is a deliberate combination, with monitoring and governance designed in from the start.

Case 1 — Customer refund assistant

Problem. Support representatives spend hours answering “is order #X refundable?” Answers are inconsistent, the wrong policy version gets quoted, and slow replies hurt satisfaction on high-value enterprise accounts.

Solution (across the stack). Retrieve the current refund policy (L4) and call the order system for live status (L5); the model composes a grounded, cited answer (L3) in chat (L1); refunds above a threshold are routed through an approval gate (L2).

Monitoring & maintenance. Check faithfulness — does the answer match the cited policy? Re-index on every policy version change and alert on stale sources. Track the order-lookup tool’s error rate and latency.

Governance. Refunds over a set amount require human approval before execution; every quoted policy and tool call is logged for audit; PII is minimized in prompts; an evaluation set gates each release.

Case 2 — HR policy assistant

Problem. HR is flooded with repetitive policy questions; answers vary by who responds; some questions touch sensitive records. Employees want self-service, but the PII risk is real.

Solution (across the stack). Retrieval-augmented Q&A over the HR policy corpus (L4), grounded in the model (L3); chat for employees (L1) with a permissioned lookup of the asker’s *own* balances (L5); “submit leave” is a write action placed behind an approval gate (L2).

Monitoring & maintenance. Track retrieval quality on a curated policy Q&A set; watch for prompt injection via uploaded policy documents; flag low-confidence answers for human follow-up.

Governance. RBAC ensures the agent cannot read records beyond the requester; a human gate guards all write actions; full reasoning traces are logged; EU AI Act transparency obligations are met; the build is red-teamed periodically.

Case 3 — Monthly variance-report agent

Problem. Analysts manually pull data from several systems, reconcile it by hand, and draft the monthly variance report. The process is slow, error-prone, and leaves no consistent trail of how figures were derived.

Solution (across the stack). An orchestrator plans the steps (L2); it queries the warehouse via a SQL tool (L6/L5), retrieves prior commentary (L4), drafts the narrative with the model (L3), and delivers it in a dashboard (L1). Multi-write steps are kept atomic with two-phase commit (L2).

Monitoring & maintenance. Check numeric accuracy against the source of truth with reconciliation tolerances; verify pipeline freshness before each run (L6); cap cost and loop counts; run a quality evaluation on the drafted report.

Governance. No autonomous posting to systems of record without sign-off; the full reasoning and tool trace is retained for auditors; data access is scoped; the model and version are pinned per run for reproducibility.

Part 10 — Live demos (run them yourself)

The deck is the show; five **progressive live demos** let participants *do* and *observe*. They run locally in Docker — one command, `docker compose up`, then open `http://localhost:8501` and paste the workshop OpenAI key into the sidebar. Each level adds exactly one capability and lights up more of the 7-layer stack, so the progression itself teaches the architecture. (Facilitator setup and management: see the separate *Live Demos — Facilitator Guide*.)

Level 1 — Chatbot (Layers 1, 3). A system prompt plus a single message, and nothing else: no memory, no guardrails. It shows exactly what is sent to the model, and makes the point that a bare chatbot is just configuration (the system prompt) over a model call. Observe: change the system prompt and watch behavior change; send a second message and notice it doesn’t remember the first.

Level 2 — Memory + guardrails (Layers 1, 3, 7). The same chat, now scoped to a narrow support task (a “Northwind Cloud” support bot) with two visible additions: conversation **memory** (the running history, shown in an expander, is replayed to the model each turn) and a **guardrail**

that checks whether each message is in-scope and refuses off-topic requests. A toggle turns the guardrail off so participants see the bot wander without it. This is where governance (Layer 7) first appears, in miniature.

Level 3 — Context engineering (Layers 4, 6). A question is answered by first **retrieving** from an information store and then **engineering the context** — the demo shows the retrieved chunks, the exact assembled prompt, and the grounded, cited answer, with an option to show the ungrounded answer side by side. The lesson lands visually: grounding is an information-retrieval and data problem, and retrieval quality dominates output quality.

Level 4 — MCP + tools (Layers 2, 5). The model gains **tools** exposed by an MCP-style server (`get_order`, `search_kb`, `calculator`). The demo renders the server’s tool catalog and then the client↔server message flow — each tool call as a request and response — before the final answer. The agent can now *act*, not just talk. The real MCP protocol over Docker lives in the repo’s `mcp-lab/` for anyone who wants the deeper version.

Level 5 — A2A + governance (Layers 2, 7). A refund request is handled by **multiple agents** — an orchestrator, a read-only research agent, and a write-capable action agent — exchanging agent-to-agent messages. Every step runs under governance: **RBAC** (only the action agent may issue a refund; a read-side write attempt is blocked in code), a **human approval gate** before any write executes, and an **append-only audit log** of every message, tool call, and decision. This is the whole-stack capstone: autonomy made safe with the controls from Layer 7.

Teaching note. Walk the levels in order during the session and narrate what each one adds; then point participants to run them between sessions. The demos deliberately mirror the deck’s running examples (refund assistant, support bot) so the concepts compound.

Appendix — How this maps to the two MS programs

The stack doubles as a coverage map for the two new degrees. Approximate alignment:

- **Layer 1 Experience, Layer 3 Model** → *Generative AI & Enterprise Applications, Adaptive AI Systems*.
- **Layer 2 Orchestration** → *Agentic AI & Business Process Design*.
- **Layer 4 Retrieval & context, Layer 6 Data foundation** → *Building AI Data Pipelines, Business Data Foundations for AI, Data Strategy, Quality & Platform Architectures*.
- **Layer 5 Enterprise systems** → *Enterprise AI Systems Architecture, Enterprise Architecture for Scalable AI Deployment*.
- **Layer 7 Governance & observability** → *Enterprise AI Risk & Governance, AI Security, Privacy & Governance*.
- **The cases** → integrative capstones (*Architecting & Leading AI Digital Transformation, Integrating AI into the Enterprise*), which require exactly this whole-stack reasoning.

Used this way, the half-day is a teaser: each layer can expand into its own deeper, hands-on session across the fall and spring, landing faculty right before they teach the corresponding course.

References

1. RAND Corporation (2024). *The Root Causes of Failure for Artificial Intelligence Projects and How They Can Succeed: Avoiding the Anti-Patterns of AI* (RR-A2680-1). J. Ryseff, B. F. De Bruhl, S. J. Newberry. https://www.rand.org/pubs/research_reports/RR-A2680-1.html — basis for “more than 80% of AI projects fail to reach production, about twice the non-AI IT rate, with inadequate data a recurring root cause.” Findings are from structured interviews with 65 data scientists and engineers.
2. OWASP. *Top 10 for Large Language Model Applications*. <https://genai.owasp.org> — the failure-mode taxonomy referenced in Layer 7.
3. NIST (2023). *AI Risk Management Framework (AI RMF 1.0)*.
4. ISO/IEC 42001:2023. *Information technology — Artificial intelligence — Management system*.
5. EU AI Act — Regulation (EU) 2024/1689.
6. University of South Florida. Draft MS program documents — *MS AI in Business & Digital Transformation* and *MS AI in Business & Enterprise Integration* (pre-publication) — basis for all course mappings.

A note on other statements: claims such as “the hard part is production, not the demo,” the value mechanisms, and the layer descriptions are framings and definitions rather than empirical measurements, and are presented as such.